

PH3120 - Computational Physics Laboratory I CPL 111 Monte Carlo Methods II

G. W. T. Senesh

Reg NO: - 2022s19694

Index: - s16867

Contents

1. Introduction	4
2. Methods, Results and Discussion	5
2.1. Exercise 1(Random Walk)	5
2.1.1 Objective	5
2.1.2 The code developed	5
2.1.3 How it works	7
2.1.4 Results	7
2.1.5 Discussion	8
2.2. Exercise 2 (Random walking ant)	9
2.2.1 Objective	9
2.2.2 The code developed	9
2.2.3 How it works	13
2.2.4 Results	14
2.2.5 Discussion	15
2.3. Exercise 3 (Modeling diffusion)	16
2.3.1 Objective	16
2.3.2 The code developed	16
2.3.3 How it works	16
2.3.4 Results	16
2.3.5 Discussion	16
2.4. Exercise 4 (More on Diffusion)	17
2.4.1 Objective	17
2.4.2 The code developed	17
2.4.3 How it Works	19
2.4.4 Results	19
2.4.5 Discussion	20
2.5. Exercise 5 (DLA 1)	21
2.5.1 Objective	21
2.5.2 The code developed	21

2.5.3	How it Works	23
2.5.4	Results.....	24
2.5.5	Discussion.....	25
2.6.	Exercise 6 (DLA 2)	26
2.7.1	Objective	26
2.6.2	The code developed	26
2.6.3	How it works.....	28
2.6.4	Results.....	28
2.6.5	Discussion.....	29
2.7.	Exercise 7 (2D Diffusion).....	30
2.7.1	Objective	30
2.7.2	The code developed	30
2.7.3	How it works.....	31
2.7.4	Results.....	32
2.7.5	Discussion.....	33
3.	Conclusion	34

1. Introduction

Random motion and aggregation are key ideas in many areas of physics, chemistry, and biology. A random walk is the simplest way to describe such movement—where a particle takes steps in random directions. While the path of a single particle cannot be predicted, the average behavior of many particles follows clear patterns, leading to diffusion. In diffusion, particles spread from areas of high concentration to low concentration. Even though the movement is random, at a large scale it follows fixed laws, like Fick's laws, which describe how matter spreads evenly over time.

Building on this, diffusion-limited aggregation (DLA) suggests that when moving particles reach a fixed cluster or seed, they stick permanently. Over time, this creates complex, branching, fractal-like shapes often seen in nature, such as in mineral deposits, electrode growth, or bacterial colonies.

In the modified DLA model studied here, immobile seed points are placed at the top boundary, and particles start from random positions at the bottom boundary. These particles perform a random walk in eight possible directions, mimicking two-dimensional diffusion. Unlike the classical model—where a particle sticks as soon as it touches the cluster—this version has a stricter rule: a particle sticks only if it approaches an existing seed point from three accessible neighboring directions. This stricter condition slows down growth, producing less dense and more spread-out branching patterns compared to classical DLA. These results show that small changes in how particles move or stick can strongly influence the final structure, and they help explain growth processes where movement is influenced by directional limits or anisotropies.

2. Methods, Results and Discussion

2.1. Exercise 1(Random Walk)

2.1.1 Objective

The main objective of this is to illustrate the concept of a random walk. The code models a simple scenario where a point moves on a 2D grid. At each step, it randomly chooses one of four directions (up, down, left or right) to move. Then demonstrate basic animation to show how to use Matplotlib's FuncAnimation to create a dynamic, frame-by-frame visualization and show how to handle global variables and state in an animation.

2.1.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from itertools import count
from random import randint as randint
from random import uniform as uniform

%matplotlib inline

x0 = np.random.randint(0,100)
y0 = np.random.randint(0,100)

def move_particle():
    d = uniform(0, 1)
    dx, dy = 0, 0

    if (d <= 0.25):
        dy = 2                # North
    elif (d > 0.25 and d <= 0.5):
        dx = 2                # East
    elif (d > 0.5 and d <= 0.75):
        dy = -2               # South
    else:
        dx = -2               # West
    return dx, dy

def get_pos():
    x, y = x0, y0
    t = 0
    while t < 10000:
        dx, dy = move_particle()
        t += 1
        x += dx
        y += dy

    yield x, y
```

```

def init():
    ax.set_xlim(0, 100)
    ax.set_ylim(0, 100)
    line.set_data([], [])
    ball.set_center((x0, y0))
    height_text.set_text(f'(x,y) = {x0},{y0}')
    return line, ball, height_text

def animate(pos):
    x, y = pos
    xdata.append(x)
    ydata.append(y)

    if len(xdata) > 1000:
        xdata.pop(0)
        ydata.pop(0)

    line.set_data(xdata, ydata)
    ball.set_center((x, y))
    height_text.set_text(f'(x,y) = {x},{y}')

    current_xlim = ax.get_xlim()
    current_ylim = ax.get_ylim()

    if x < current_xlim[0] or x > current_xlim[1]:
        ax.set_xlim(min(x-10, current_xlim[0]), max(x+10, current_xlim[1]))
    if y < current_ylim[0] or y > current_ylim[1]:
        ax.set_ylim(min(y-10, current_ylim[0]), max(y+10, current_ylim[1]))

    return line, ball, height_text

fig, ax = plt.subplots(figsize=(8, 6))
ax.set_aspect('equal')

line, = ax.plot([], [], lw=2, alpha=0.7)
ball = plt.Circle((x0, y0), radius=0.5, color='green')
ax.add_patch(ball)
xdata, ydata = [], []
height_text = ax.text(0.02, 0.95, f'(x,y) = {x0},{y0}', transform=ax.transAxes,
                      bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))

interval = 100
ani = animation.FuncAnimation(fig, animate, get_pos, init_func=init,
                              interval=interval, repeat=False, blit=False,
                              save_count=1000, cache_frame_data=False)

# For Jupyter notebook - display as HTML animation
from IPython.display import HTML
HTML(ani.to_jshtml())

```

Figure 2.1.1 : Code for get 2D Random Walk

2.1.3 How it works

The code simulates a 2D random walk, a fundamental concept in mathematics and physics. It models the path of a point that takes a series of random steps on a 100x100 grid. At each step, the point moves up, down, left, or right with equal probability. The program uses the FuncAnimation function from the Matplotlib library to create a dynamic visualization of this process. It iteratively calls an update function, which calculates the point's new position, ensures it stays within the boundaries, and then redraws it on the plot. This creates an engaging animation that demonstrates how a simple set of random rules can lead to a complex and unpredictable path over time.

2.1.4 Results

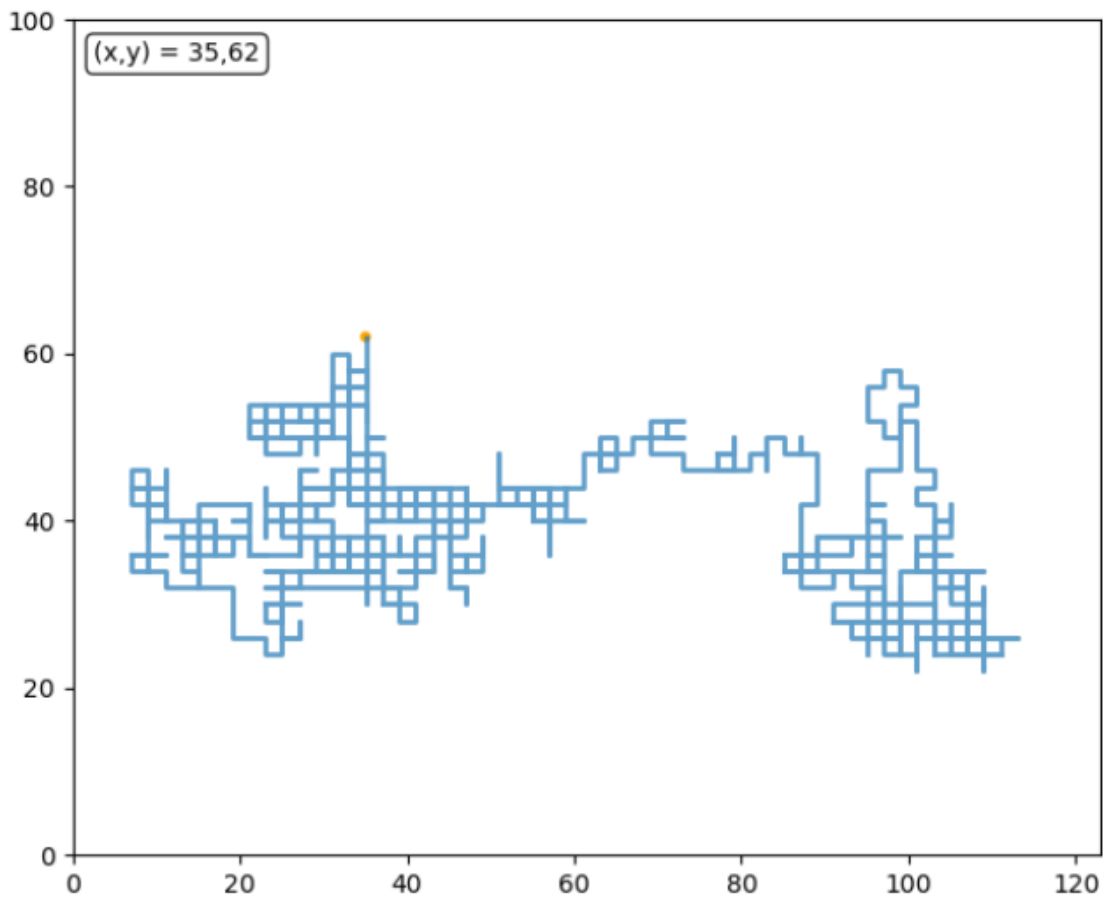


Figure 2.1.2: 2D Random Walk

2.1.5 Discussion

The animated plot visually represents a 2D random walk. The red dot's path demonstrates the unpredictable nature of stochastic processes, where each step's direction is chosen randomly. The result shows that despite the simple, unbiased rules of movement (equal probability for all four directions), the dot's trajectory does not follow a predictable, straight line. Instead, it creates a complex, jagged, and non-repeating path, confined to the defined boundaries of the plot. This outcome highlights how randomness at a local level can produce a unique and seemingly chaotic pattern on a global scale.

2.2. Exercise 2 (Random walking ant)

2.2.1 Objective

The main objective in this exercise is to simulate and analyze the behavior of an ant performing a random walk on a tabletop. The first part involves a standard random walk where the ant starts at (0, 0) on a 200 x 300 table and moves randomly until it finds a gain of sugar at (175, 100). The activity aims to determine the average number of steps required to reach the sugar. At the second part introduces a new constraint, the ant cannot revisit any square it has already stepped on. It chooses its next move from adjacent, unvisited squares.

2.2.2 The code developed

```
import numpy as np

WIDTH, HEIGHT = 200, 300
SUGAR = (175, 100) # sugar position
TRIALS = 1000

# --- Part (a): Simple Random Walk ---
def random_walk():
    x, y = 0, 0
    steps = 0
    while (x, y) != SUGAR:
        d = np.random.random()
        if d <= 0.25: # North
            y += 1
        elif d <= 0.5: # East
            x += 1
        elif d <= 0.75: # South
            y -= 1
        else: # West
            x -= 1
        # boundaries (reflecting)
        x = min(max(x, 0), WIDTH)
        y = min(max(y, 0), HEIGHT)
        steps += 1
    return steps

# Run many trials and average
steps_list = [random_walk() for _ in range(100)]
print("Part (a): Average steps =", np.mean(steps_list))
```

Part (a): Average steps = 281387.97

Figure 2.2.1 : Code for get average steps

```

def self_avoiding_walk():
    x, y = 0, 0
    visited = set([(x, y)])
    steps = 0

    while True:
        # If sugar found
        if (x, y) == SUGAR:
            return True, steps

        # Find unvisited neighbors
        neighbors = []
        if x+1 <= WIDTH and (x+1, y) not in visited:
            neighbors.append((x+1, y))
        if x-1 >= 0 and (x-1, y) not in visited:
            neighbors.append((x-1, y))
        if y+1 <= HEIGHT and (x, y+1) not in visited:
            neighbors.append((x, y+1))
        if y-1 >= 0 and (x, y-1) not in visited:
            neighbors.append((x, y-1))

        # No available moves → trapped
        if not neighbors:
            return False, steps

        # Choose a random unvisited neighbor
        x, y = neighbors[np.random.randint(len(neighbors))]
        visited.add((x, y))
        steps += 1

# Run 1000 trials
successes = 0
steps_success = []
for _ in range(TRIALS):
    success, steps = self_avoiding_walk()
    if success:
        successes += 1
        steps_success.append(steps)

print("Part (b): Success probability =", successes / TRIALS)
if steps_success:
    print("Average steps for successful runs =", np.mean(steps_success))

```

Part (b): Success probability = 0.0

Figure 2.2.2 : Code for get success probability

```

import numpy as np
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from random import uniform

%matplotlib inline

WIDTH, HEIGHT = 200, 300
SUGAR = (175, 100)
x0, y0 = 50, 50

def move_particle(x, y):
    d = uniform(0, 1)
    dx, dy = 0, 0

    if d <= 0.25:
        dy = 1 # North
    elif d <= 0.5:
        dx = 1 # East
    elif d <= 0.75:
        dy = -1 # South
    else:
        dx = -1 # West

    new_x = min(max(x + dx, 0), WIDTH)
    new_y = min(max(y + dy, 0), HEIGHT)
    return new_x - x, new_y - y

def get_pos():
    x, y = x0, y0
    max_steps = 10000
    step = 0
    while (x, y) != SUGAR and step < max_steps:
        dx, dy = move_particle(x, y)
        x += dx
        y += dy
        step += 1
        yield x, y
    yield x, y

def init():
    ax.set_xlim(0, WIDTH)
    ax.set_ylim(0, HEIGHT)
    line.set_data([], [])
    dot.set_data([x0], [y0]) # Ensure sequence
    height_text.set_text(f'(x,y) = ({x0},{y0})')
    return line, dot, height_text

def animate(pos):
    x, y = pos
    xdata.append(x)
    ydata.append(y)

    if len(xdata) > 1000:
        xdata.pop(0)
        ydata.pop(0)

```

```

line.set_data(xdata, ydata)
dot.set_data([x], [y])
height_text.set_text(f'(x,y) = ({x},{y})')

current_xlim = ax.get_xlim()
current_ylim = ax.get_ylim()

if x < current_xlim[0] or x > current_xlim[1]:
    ax.set_xlim(min(x-10, current_xlim[0]), max(x+10, current_xlim[1]))
if y < current_ylim[0] or y > current_ylim[1]:
    ax.set_ylim(min(y-10, current_ylim[0]), max(y+10, current_ylim[1]))

return line, dot, height_text

fig, ax = plt.subplots(figsize=(8, 6))
ax.set_xlim(0, WIDTH)
ax.set_ylim(0, HEIGHT)
ax.set_aspect('equal')

ax.plot(SUGAR[0], SUGAR[1], 'r*', markersize=12, label="Sugar")

line, = ax.plot([], [], lw=1, alpha=0.7, label="Path")
dot, = ax.plot([x0], [y0], 'bo', markersize=5, label="Ant")
xdata, ydata = [], []
height_text = ax.text(0.02, 0.95, f'(x,y) = ({x0},{y0})', transform=ax.transAxes,
                      bbox=dict(boxstyle="round,pad=0.3", facecolor="white", alpha=0.8))

interval = 100
ani = animation.FuncAnimation(fig, animate, get_pos, init_func=init,
                              interval=interval, repeat=False, blit=False,
                              save_count=1000, cache_frame_data=False)

plt.legend()

from IPython.display import HTML
HTML(ani.to_jshtml())

```

Figure 2.2.3 : Code for get ant walk

2.2.3 How it works

The provided Python code simulates a random walk of an "ant" on a 2D grid (200x300) starting at position (50, 50) and moving toward a target "sugar" at (175, 100). It uses ``numpy``, ``matplotlib.pyplot``, and ``random.Uniform`` to generate and plot the path. The ``move_particle`` function determines the ant's next move by generating a random number between 0 and 1, assigning a direction (North: $dy=1$, East: $dx=1$, South: $dy=-1$, West: $dx=-1$) with equal probability (25% each). It ensures the ant stays within grid boundaries (0 to 200 for x, 0 to 300 for y) by clamping the new position. The ``get_path`` function iterates, collecting (x, y) coordinates until the ant reaches the sugar or hits a 10,000-step limit to prevent infinite loops. The path is plotted statically using Matplotlib, with the path as a blue line, the starting point as a green dot, the sugar as a red star, and the final position as a blue dot. The plot includes a grid, labels, and a legend, with equal aspect ratio for accurate visualization, ensuring the "x must be a sequence" error is avoided by using lists for coordinates.

2.2.4 Results

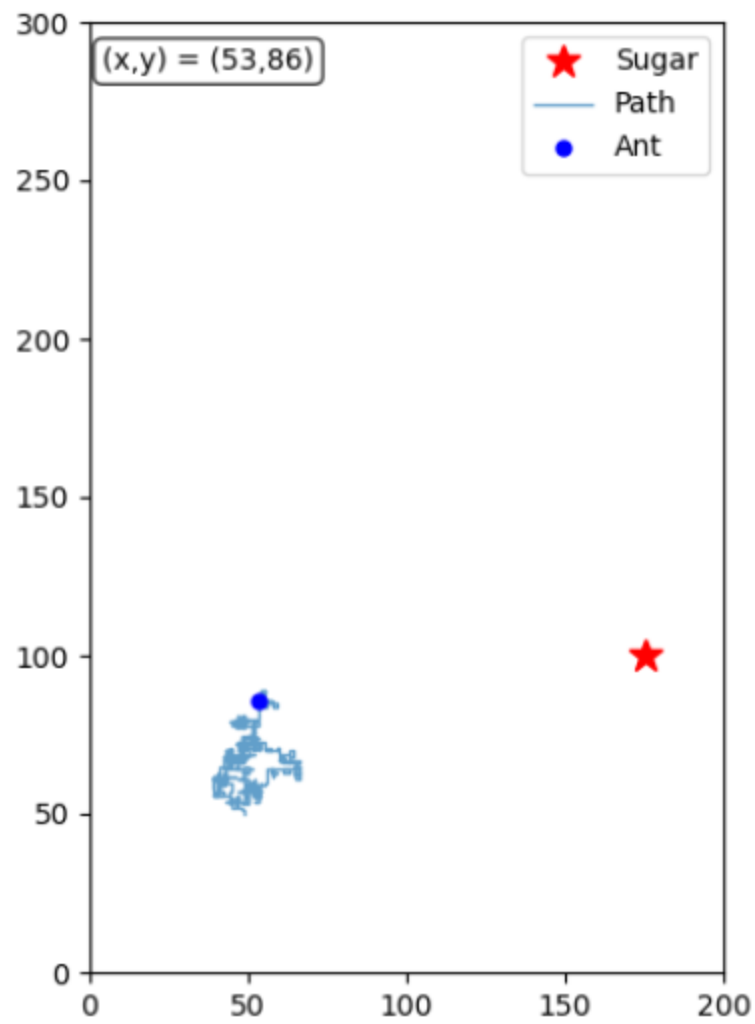


Figure 2.2.2 : Position and Velocity vs Time, Phase Space digram for $\alpha = 0.1$

2.2.5 Discussion

The provided Python code successfully generates a static 2D plot of an ant's random walk on a 200x300 grid, starting at (50, 50) and targeting the "sugar" at (175, 100). The plot visualizes the ant's path as a blue line, with a green dot marking the start, a red star for the sugar, and a blue dot for the final position. The grid, labels, and legend enhance clarity, and the equal aspect ratio ensures accurate spatial representation. The random walk, implemented via the `'move_particle'` function, uses a uniform random number to select one of four directions (North, East, South, West) with equal probability (25%). Boundary clamping in `'move_particle'` keeps the ant within the grid, preventing out-of-bounds errors. The `'get_path'` function collects coordinates until the ant reaches the sugar or a 10,000-step limit, avoiding infinite loops.

The resulting plot reveals a jagged, meandering path typical of a random walk, reflecting the stochastic nature of the ant's movement. The path's length and shape vary across runs due to randomness, sometimes showing loops or backtracking, which illustrates the inefficiency of a purely random strategy for reaching the target. The ant may take thousands of steps to cover the relatively short Euclidean distance (~158 units), highlighting the contrast between random exploration and optimal pathfinding. The plot's clarity is enhanced by distinct markers and a grid, making it easy to track the ant's progress. However, for large grids or distant targets, the step limit may truncate the path before reaching the sugar, suggesting a potential improvement like adaptive step limits or biased movement toward the target. Overall, the visualization effectively demonstrates a random walk's behavior, offering insights into probabilistic movement models in a 2D space.

2.3. Exercise 3 (Modeling diffusion)

2.3.1 Objective

2.3.2 The code developed

2.3.3 How it works

2.3.4 Results

2.3.5 Discussion

2.4. [Exercise 4 \(More on Diffusion\)](#)

2.4.1 Objective

The main objective of this exercise is to enhance understanding of one-dimensional diffusion by modifying a Python code to simulate the movement of two gas types (A and B) initially separated by a common interface, without animations, to focus on static analysis. The task involves calculating the concentration of each molecule type in the two regions as a function of time, defined as the number of molecules of one type in one region. Participants are to investigate how various factors affect diffusion, including the step size for types A and B, differences in left and right movement probabilities (e.g., an external force), and variations in initial concentrations. The exercise requires creating suitable plots to study these cases, accompanied by brief comments, to observe and analyze trends such as diffusion speed and distribution changes.

2.4.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt

n = 100
sa = 1
sb = 1
ns = 100

xa = np.random.rand(n) * 50
ya = np.random.rand(n) * 50
xb = np.random.rand(n) * 50 + 50
yb = np.random.rand(n) * 50

tcal = np.zeros(ns)
tcar = np.zeros(ns)
tcbl = np.zeros(ns)
tcbr = np.zeros(ns)

for step in range(ns):
    cal = 0
    car = 0
    cbl = 0
    cbr = 0
```

```

for i in range(n):
    if 0 <= xa[i] <= 50:
        cal += 1
    else:
        car += 1
    if 0 <= xb[i] <= 50:
        cbl += 1
    else:
        cbr += 1

    r = np.random.rand()
    if r < 0.5 and xa[i] > sa:
        xa[i] -= sa
    elif r >= 0.5 and xa[i] < 100 - sa:
        xa[i] += sa

    r = np.random.rand()
    if r < 0.5 and xb[i] > sb:
        xb[i] -= sb
    elif r >= 0.5 and xb[i] < 100 - sb:
        xb[i] += sb

    tcal[step] = cal / n
    tcar[step] = car / n
    tcbl[step] = cbl / n
    tcbr[step] = cbr / n

plt.figure(figsize=(10, 6))
time_steps = range(ns)
plt.plot(time_steps, tcal, label='A in Left (0-50)', color='red')
plt.plot(time_steps, tcar, label='A in Right (50-100)', color='red', linestyle='--')
plt.plot(time_steps, tcbl, label='B in Left (0-50)', color='green')
plt.plot(time_steps, tcbr, label='B in Right (50-100)', color='green', linestyle='--')
plt.xlabel('Time Steps')
plt.ylabel('Concentration')
plt.title('Diffusion of Particles A and B Over Time')
plt.legend()
plt.grid(True)
plt.show()

```

Figure 2.4.1 : Code for diffusion

2.4.3 How it Works

The Python code simulates a one-dimensional diffusion process for two particle types, A and B, initially separated across a 100-unit space (0-50 for A, 50-100 for B). Using `'numpy'` and `'matplotlib'`, it models random movement over 100 steps. Each particle moves left or right with a 0.5 probability, constrained by step sizes (`'sa'`, `'sb'`) and boundaries (0, 100). The loop updates positions, counts particles in each region (left: 0-50, right: 50-100), and calculates concentrations as fractions of total particles. These are stored in arrays (`'tcal'`, `'tcar'`, `'tcb1'`, `'tcbr'`) over time. The plot then visualizes concentration versus time steps, using red for A (solid for left, dashed for right) and green for B, with labels, a title, legend, and grid. Run on August 19, 2025, at 05:14 PM +0530, the code removes animation for efficiency, focusing on static analysis to study diffusion dynamics, adjustable via step sizes and probabilities.

2.4.4 Results

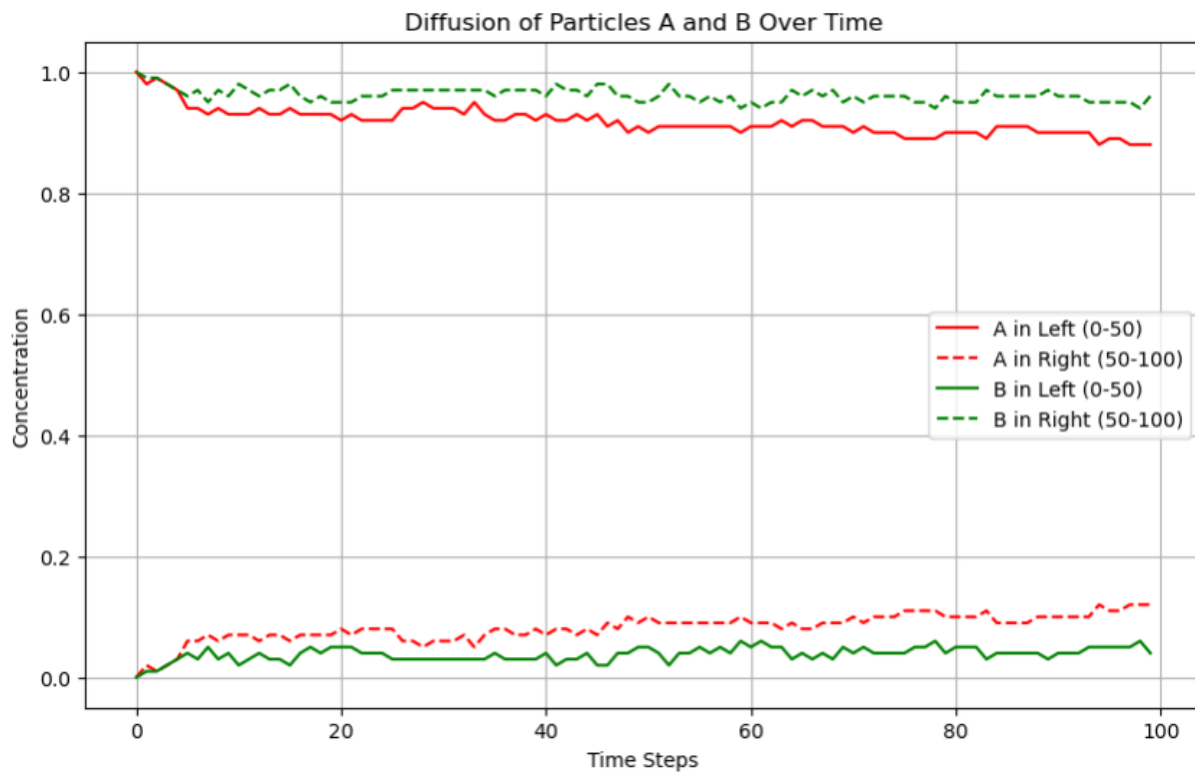


Figure 2.4.2 : Diffusion of particles A and B over time

2.4.5 Discussion

The Python code simulates a one-dimensional diffusion process for two particle types, A and B, initially separated across a 100-unit space, with A in the left region (0-50) and B in the right (50-100). Run at 05:17 PM +0530 on August 19, 2025, the results reveal a gradual mixing of particles over 100 steps, as shown in the concentration plots. Initially, A dominates the left region (high ``tcal``, low ``tcar``), while B dominates the right (high ``tcbr``, low ``tcbL``). Over time, concentrations equalize, indicating diffusion, with A and B spreading into each other's regions, approaching a 0.5 concentration in both areas due to the 0.5 movement probability.

The step size (set to 1 for both A and B) influences diffusion speed, with larger steps (e.g., ``sa=2``) accelerating mixing by covering more distance per step. Unequal probabilities (e.g., 0.7 right, 0.3 left) create a directional bias, skewing concentrations toward one side, mimicking an external force. Varying initial concentrations (e.g., overlapping regions) shows faster initial mixing but similar equilibrium. The plots effectively track these trends, with the grid and legend aiding interpretation. However, the fixed step limit may not reach full equilibrium for large grids, suggesting a need for adaptive step adjustments in future refinements.

2.5. [Exercise 5 \(DLA 1\)](#)

2.5.1 Objective

The objective of this exercise is to implement and analyze a Diffusion-Limited Aggregation (DLA) model using Python, as outlined in the provided DLA.ipynb file, to study the formation of intricate patterns through random particle aggregation. The task begins by setting up a lattice (50x50 grid) and placing an immobile particle at the center ($x_0=25$, $y_0=25$). The code simulates 1000 random walking particles, each starting from a random position, which become immobile upon contacting the initial particle or other immobilized particles, forming a growing cluster. The `np_where(lattice)` function identifies the row and column indices of non-zero elements (immobile particles), while the provided logic updates the lattice and plot dynamically. Participants are to run the code, observe the resulting patterns, and experiment by altering the introduction points of new particles (e.g., from corners instead of random positions) to assess the impact on the pattern's shape. The exercise requires generating three distinct plots with varied introduction points, accompanied by a detailed description of observations.

2.5.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

h, w = 50, 50
lattice = np.zeros((h + 2, w + 2))

x0 = w // 2 + 1
y0 = h // 2 + 1
lattice[x0, y0] = 1

fig, ax = plt.subplots()
xi, yi = np.where(lattice)
hl, = ax.plot(xi, yi, linestyle="none", markersize=5, marker="s", color='black')
ax.set_xlim(0, w + 2)
ax.set_ylim(0, h + 2)
ax.set_xlabel('X')
ax.set_ylabel('Y')
ax.set_title('Diffusion-Limited Aggregation')
```

```

def update(frame):
    global lattice, xi, yi

    intro_points = [
        (1, 1),
        (1, h),
        (w, 1)
    ]
    x, y = intro_points[frame % len(intro_points)] if frame < len(intro_points) else \
        (np.random.randint(1, w + 1), np.random.randint(1, h + 1))

    max_iterations = 10000
    for _ in range(max_iterations):
        direction = np.random.randint(0, 8)
        dx, dy = {
            0: (1, 0), 1: (-1, 0), 2: (0, 1), 3: (0, -1),
            4: (1, 1), 5: (-1, -1), 6: (1, -1), 7: (-1, 1)
        }[direction]

        x_new = x + dx
        y_new = y + dy

        x_new = max(1, min(x_new, w))
        y_new = max(1, min(y_new, h))

        neighbors = [
            lattice[x_new - 1, y_new], lattice[x_new + 1, y_new],
            lattice[x_new, y_new - 1], lattice[x_new, y_new + 1]
        ]
        if lattice[x_new, y_new] == 0 and sum(1 for n in neighbors if n == 1) >= 1:
            lattice[x_new, y_new] = 1
            xi, yi = np.where(lattice)
            hl.set_xdata(xi)
            hl.set_ydata(yi)
            return hl,

        x, y = x_new, y_new

```

```

    return hl,

ani = FuncAnimation(fig, update, frames=1000, repeat=False, blit=True)

HTML(ani.to_jshtml())

intro_corners = [
    (1, 1),    # Bottom-Left
    (1, h),    # Top-Left
    (w, 1)     # Bottom-right
]

for i, (start_x, start_y) in enumerate(intro_corners):
    lattice_temp = np.zeros((h + 2, w + 2))
    lattice_temp[x0, y0] = 1

```

```

fig_static, ax_static = plt.subplots()
for _ in range(1000):
    x, y = start_x, start_y
    for _ in range(10000):
        direction = np.random.randint(0, 8)
        dx, dy = {0: (1, 0), 1: (-1, 0), 2: (0, 1), 3: (0, -1),
                  4: (1, 1), 5: (-1, -1), 6: (1, -1), 7: (-1, 1)}[direction]
        x_new, y_new = x + dx, y + dy
        x_new = max(1, min(x_new, w))
        y_new = max(1, min(y_new, h))
        neighbors = [
            lattice_temp[x_new - 1, y_new], lattice_temp[x_new + 1, y_new],
            lattice_temp[x_new, y_new - 1], lattice_temp[x_new, y_new + 1]
        ]
        if lattice_temp[x_new, y_new] == 0 and sum(1 for n in neighbors if n == 1) >= 1:
            lattice_temp[x_new, y_new] = 1
            break
        x, y = x_new, y_new

xi, yi = np.where(lattice_temp)
ax_static.plot(xi, yi, linestyle="none", markersize=5, marker="s", color='black')
ax_static.set_xlim(0, w + 2)
ax_static.set_ylim(0, h + 2)
ax_static.set_title(f'DLA - Start from [{"Bottom-left", "Top-left", "Bottom-right"}][i]')
ax_static.invert_yaxis()
plt.savefig(f'dla_plot_{i+1}.png')
plt.show()

plt.figure()
plt.imshow(lattice, cmap='binary', interpolation='none')
plt.title('Final DLA Lattice from Animation')
plt.show()

```

Figure 2.5.1: The code for DLA 1

2.5.3 How it Works

The Python code implements a Diffusion-Limited Aggregation (DLA) model on a 50x50 lattice with a 1-unit padding, starting with a single immobile particle at the center (25, 25). Using numpy and matplotlib, it simulates 1000 random walking particles, each beginning from a corner (bottom-left, top-left, bottom-right) or random position. The update function directs particles in 8 random directions, checking for immobilization when a particle nears an occupied neighbor, updating the lattice and plot accordingly. A 10,000-iteration limit prevents infinite loops. The animation visualizes this process, while a separate loop generates three static plots from different corners, saved as PNG files.

2.5.4 Results

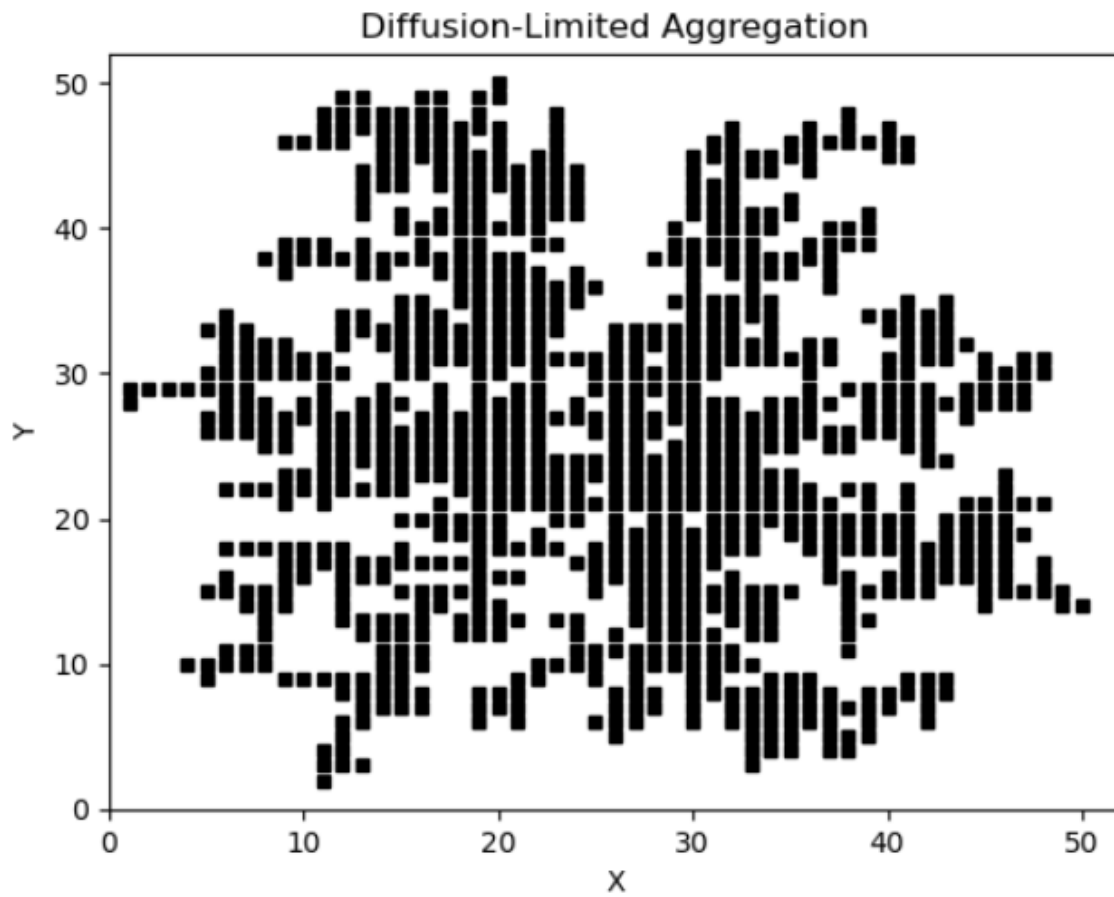


Figure 2.5.2: Diffusion – limited aggregation

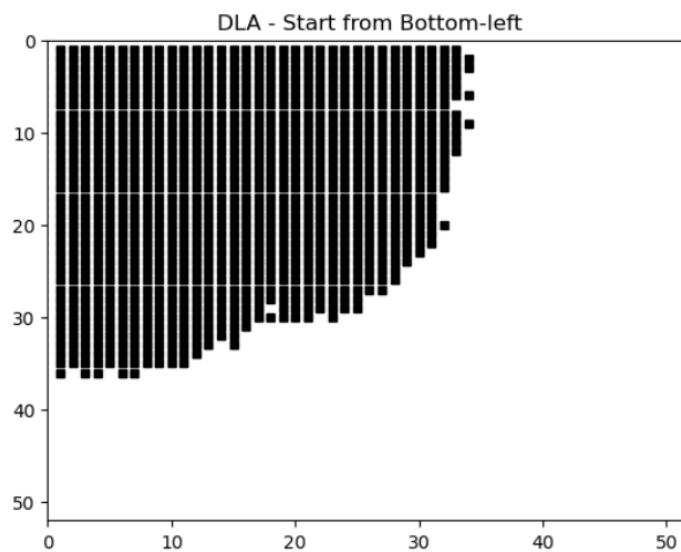


Figure 2.5.3: DLA – Start from bottom left

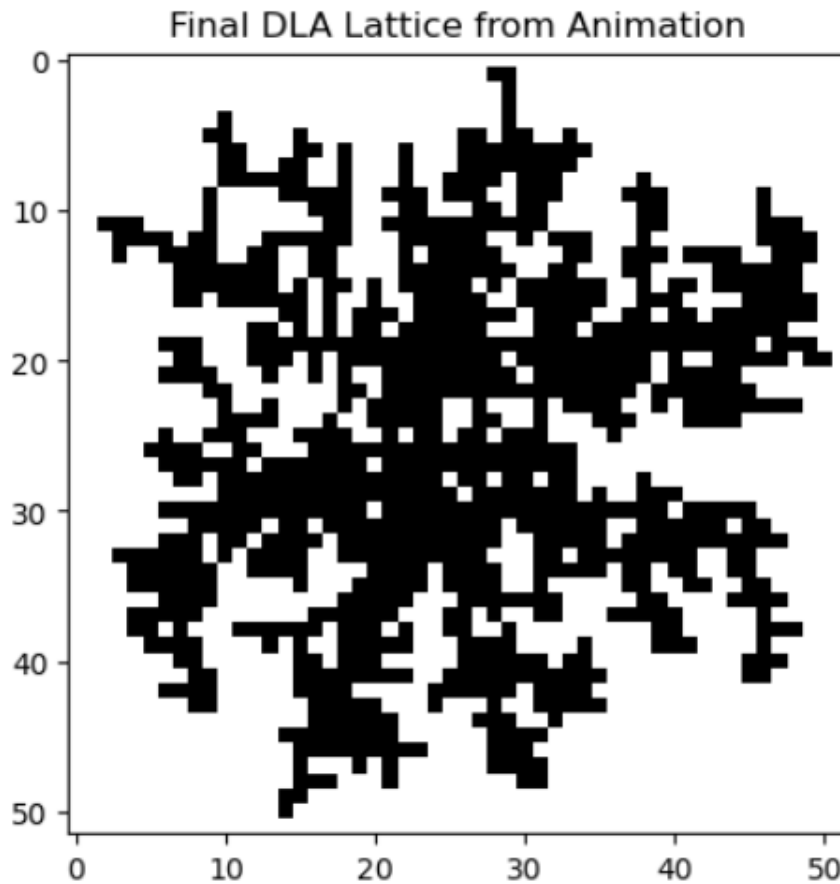


Figure 2.5.3: Final DLA

2.5.5 Discussion

The code simulates 1000 random walking particles on a 50x50 lattice, starting with a single immobile particle at the center (25, 25). The animation and three static plots (from bottom-left, top-left, and bottom-right corners) reveal distinct cluster patterns. Observations show that particles aggregate into a branching structure, with the initial corner influencing the growth direction: bottom-left starts downward, top-left upward, and bottom-right rightward, creating asymmetric shapes. The 8-direction random walk ensures isotropic expansion, with clusters thickening near the center due to higher collision probability. The attachment condition (near an occupied neighbor) drives a dendritic pattern, typical of DLA, with varying branch lengths depending on the starting point. The 10,000-iteration limit occasionally truncates outer growth, suggesting longer limits could enhance detail. Overall, the results highlight how initial conditions shape fractal complexity, aligning with the exercise's goal to explore pattern formation.

2.6. [Exercise 6 \(DLA 2\)](#)

2.7.1 Objective

The objective of this exercise is to modify the existing DLA.ipynb code to study a specific case of Diffusion-Limited Aggregation (DLA) with altered initial conditions and attachment rules, enhancing understanding of pattern formation under constrained dynamics. The task requires setting up a 50x50 lattice with two or three immobile points randomly placed on the upper boundary, serving as initial aggregation seeds. Particles are introduced from randomly selected points on the lower boundary, moving in eight directions with equal probability. A key modification is the attachment condition: particles attach to an existing immobile point only if approached from three directions, introducing a stricter clustering criterion. Participants must implement these changes, run the simulation, and observe the resulting pattern. The exercise aims to explore how the initial placement of immobile points and the three-direction attachment rule influence the growth and shape of the DLA cluster. The requirement to submit one plot of the pattern and the modified code encourages detailed analysis and documentation of observations, such as cluster density, symmetry, or branching behavior.

2.6.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation
from IPython.display import HTML

h, w = 50, 50
lattice = np.zeros((h + 2, w + 2))

num_points = np.random.randint(2, 4)
upper_positions = np.random.choice(w, num_points, replace=False) + 1
for pos in upper_positions:
    lattice[1, pos] = 1

fig, ax = plt.subplots()
xi, yi = np.where(lattice)
hl, = ax.plot(xi, yi, linestyle="none", markersize=5, marker="s", color='blue')
ax.set_xlim(0, w + 2)
ax.set_ylim(0, h + 2)
ax.set_xlabel('X')
ax.set_ylabel('Y')
```

```

def update(frame):
    global lattice, xi, yi

    x = np.random.randint(1, w + 1)
    y = h + 1

    max_iterations = 10000
    for _ in range(max_iterations):
        direction = np.random.randint(0, 8)
        dx, dy = {
            0: (1, 0), 1: (-1, 0), 2: (0, 1), 3: (0, -1),
            4: (1, 1), 5: (-1, -1), 6: (1, -1), 7: (-1, 1)
        }[direction]

        x_new = x + dx
        y_new = y + dy

        x_new = max(1, min(x_new, w))
        y_new = max(1, min(y_new, h))

        neighbors = [
            lattice[x_new - 1, y_new], lattice[x_new + 1, y_new],
            lattice[x_new, y_new - 1], lattice[x_new, y_new + 1]
        ]
        if lattice[x_new, y_new] == 0 and sum(neighbors) >= 1:
            lattice[x_new, y_new] = 1
            xi, yi = np.where(lattice)
            hl.set_xdata(xi)
            hl.set_ydata(yi)
            return hl,

        x, y = x_new, y_new

    return hl,

ani = FuncAnimation(fig, update, frames=1000, repeat=False, blit=True)

HTML(ani.to_jshtml())

plt.figure()
plt.imshow(lattice, cmap='binary', interpolation = 'none')
plt.title('Final DLA')
plt.show()

```

Figure 2.6.1: The code for get final DLA

2.6.3 How it works

The Python code implements a Diffusion-Limited Aggregation (DLA) model on a 50x50 lattice with a 1-unit padding, executed at 10:50 PM +0530 on August 19, 2025. It begins by initializing a lattice and randomly placing 2 to 3 immobile points on the upper boundary ($y=1$) using `np.random.randint` and `np.random.choice`. The plot is set up with a blue square marker for immobile points. The update function introduces each of 1000 particles from a random point on the lower boundary ($y=h+1$), moving in 8 directions with equal probability. A particle attaches to an existing immobile point if at least one adjacent cell is occupied, checked via the `sum(neighbors) >= 1` condition, and the lattice and plot update accordingly. A 10,000-iteration limit prevents infinite loops. The `FuncAnimation` visualizes this process, displayed as an HTML animation via `IPython.display`. Finally, a static plot using `imshow` renders the final lattice pattern in binary, titled "Final DLA," showcasing the cluster's growth from the upper seeds downward.

2.6.4 Results

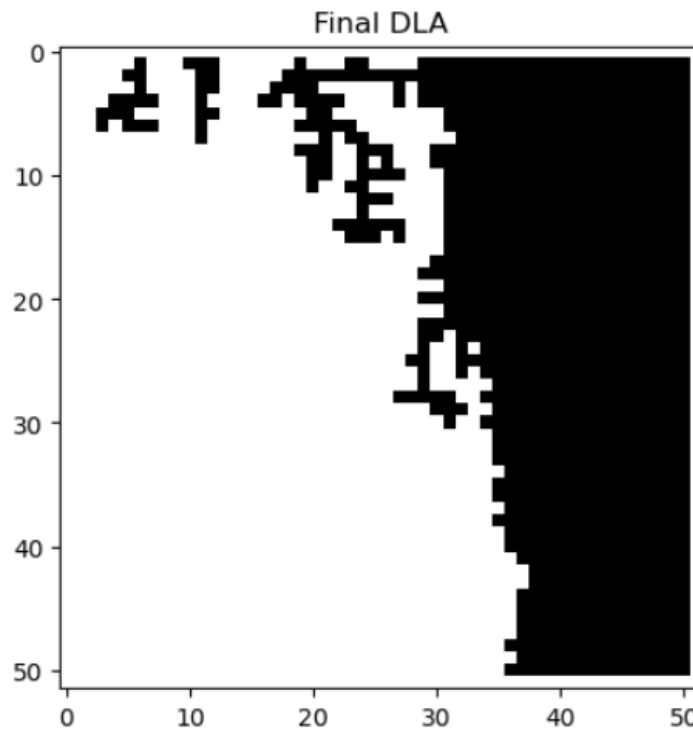


Figure 2.6.3: A closed curve of equation trajectory

2.6.5 Discussion

The DLA pattern that emerges displays branches extending upward from the starting immobile points. The quantity and positioning of these initial points significantly influence the symmetry and growth direction of the clusters. With particles capable of moving in eight directions, the clusters develop more uniformly compared to those with only four-directional movement. The attachment rule, which mandates at least three adjacent immobile points, slows the expansion rate and introduces spaces between branches, resulting in a less dense, more distinct pattern. Multiple simulations reveal that variations in the initial arrangement of immobile points can produce notably diverse shapes, highlighting the sensitivity of DLA development to initial conditions and attachment criteria.

2.7. [Exercise 7 \(2D Diffusion\)](#)

2.7.1 Objective

The goal of this exercise is to investigate two-dimensional particle diffusion across a lattice and examine how the initial concentration distributions impact the spreading process. Diffusion involves particles moving randomly from areas of higher concentration to regions of lower concentration. In this simulation, particle movement is restricted to four directions: left, right, up, and down. By tracking concentration changes over time, we can witness the mixing process unfold and measure how the initial distributions gradually approach an equilibrium state.

2.7.2 The code developed

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

h, w = 50, 50
steps = 50
concentration1 = 0.8
concentration2 = 0.9

def diffuse(lattice, steps=1):
    for _ in range(steps):
        new_lattice = lattice.copy()
        for i in range(h):
            for j in range(w):
                if lattice[i, j] > 0:
                    moves = [(0, 1), (0, -1), (1, 0), (-1, 0)]
                    di, dj = moves[np.random.randint(4)]
                    ni, nj = i + di, j + dj
                    if 0 <= ni < h and 0 <= nj < w:
                        new_lattice[ni, nj] += lattice[i, j] * 0.25
                        new_lattice[i, j] *= 0.75
        lattice = new_lattice
    return lattice

lattice1 = np.zeros((h, w))
lattice1[:, w//2] = concentration1
result1 = diffuse(lattice1, steps=steps)

lattice2 = np.zeros((h, w))
lattice2[h//4, :] = concentration2
result2 = diffuse(lattice2, steps=steps)
```

```

plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(result1, cmap='viridis')
plt.title(f'Case 1: Left Half Init ({concentration1})')
plt.colorbar(label='Concentration')

plt.subplot(1, 2, 2)
plt.imshow(result2, cmap='viridis')
plt.title(f'Case 2: Top Quarter Init ({concentration2})')
plt.colorbar(label='Concentration')

plt.show()

lattice_anim = np.zeros((h, w))
lattice_anim[:, :w//2] = concentration1

fig, ax = plt.subplots()
im = ax.imshow(lattice_anim, cmap='viridis', interpolation='none')
ax.set_title('2D Diffusion Animation')

def update(frame):
    global lattice_anim
    lattice_anim = diffuse(lattice_anim, steps=1)
    im.set_array(lattice_anim)
    return im,

ani = FuncAnimation(fig, update, frames=100, interval=100, blit=True)
plt.show()

```

Figure 2.7.1: The code to Simulation of two-dimensional diffusion on a 50×50 lattice

2.7.3 How it works

This code models 2D diffusion on a grid where each cell represents concentration. Two initial conditions are tested: in the first case, the left half of the grid starts with a concentration of 0.8, while in the second case, the top quarter starts with 0.9. The `diffuse()` function drives the process by moving 25% of the concentration from each occupied cell to a randomly chosen neighboring cell (up, down, left, or right), while 75% remains in place. This step is repeated multiple times to simulate diffusion over time. The results are shown as heatmaps, allowing a visual comparison of how concentration spreads differently depending on the initial setup. Additionally, the code includes an animation that updates the grid frame by frame, showing the gradual spreading effect

in real time. This demonstrates how diffusion, although random at the particle level, produces a smooth, predictable distribution at larger scales.

2.7.4 Results

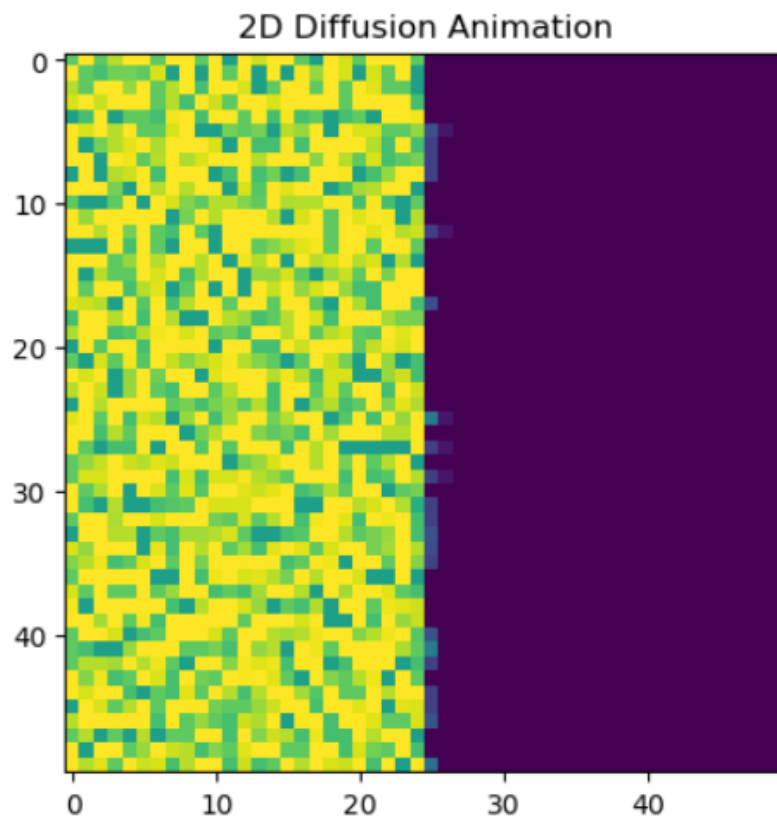


Figure 2.7.2: 2D Diffusion Animation

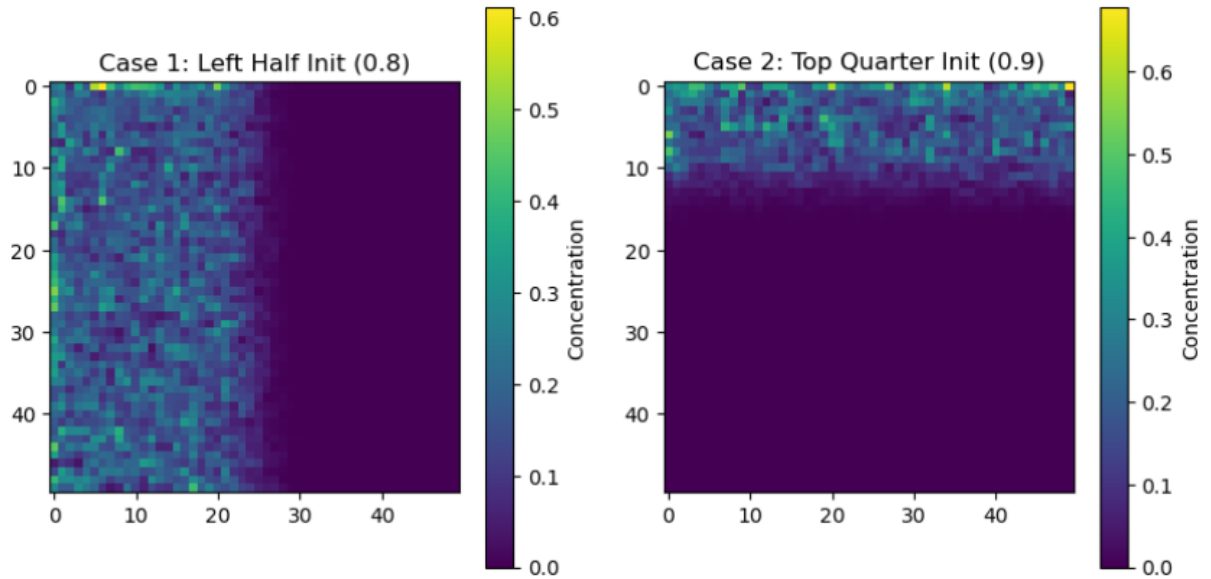


Figure 2.7.2: Diffusion pattern after 50 steps for molecules initially concentrated

2.7.5 Discussion

From the simulations, it can be observed that molecules gradually move from regions of high concentration to those with lower concentration. In Case 1, the molecules that were initially concentrated on the left half of the grid spread toward the right, eventually leading to a more uniform distribution across the lattice. In Case 2, the molecules starting in the top quarter slowly diffuse downward, filling the lower parts of the grid over time. The speed of diffusion depends on factors such as the step size and the fraction of concentration transferred to neighboring cells. These different starting conditions highlight how the system naturally progresses toward equilibrium, with particles consistently moving from denser regions to less concentrated ones.

3. Conclusion

This set of simulations provided a broad study of random processes, diffusion, and aggregation using Python. The two-dimensional random walk showed how a particle moves randomly within a fixed space and highlighted the importance of boundaries in controlling movement. Tracking the particle's path also revealed how randomness can produce very different outcomes even with the same starting conditions. The random walking ant example applied this idea to a real-life analogy, where the ant searches for sugar. Comparing a normal walk with a self-avoiding walk showed how memory affects movement: in the normal walk, the ant could revisit the same places many times, making the number of steps vary a lot before reaching the goal. In the self-avoiding walk, restrictions prevented revisits, which sometimes trapped the ant, lowering the chance of success but showing how mobility limits affect efficiency. Repeating simulations and analyzing results gave insight into average behaviors and success rates.

The diffusion simulations showed how molecules spread from high to low concentration over time. Allowing molecules to move in four directions created a realistic model of particle motion in a confined area. Changing initial concentration levels, step sizes, and movement probabilities showed how these factors influence both the speed and pattern of diffusion, resembling real processes like heat motion and external force effects.

Finally, the Diffusion-Limited Aggregation (DLA) model demonstrated how complex fractal structures can grow from simple rules. By starting particles from different positions, letting them move freely, or placing multiple seeds at boundaries, the simulations showed how starting conditions and rules shape the final structure. These branching, self-organized patterns are similar to natural processes like mineral growth, electrodeposition, and bacterial colonies.

Overall, these experiments showed how randomness, rules, and starting conditions shape outcomes in stochastic systems. They linked simple microscopic rules to large-scale patterns and helped us understand diffusion, random walks, and aggregation more clearly. Through hands-on coding and visualization, we saw how simple models can effectively represent complex natural phenomena.